



Dynamiska “/endpoints”

“ Steg #2 på föregående lektion! ”



INNEHÅLLSFÖRTECKNING

01

Översikt

03

Next.js
Frontend

02

Annotations
&
Endpoints

04

Eget arbete
&
Uppgifter

Kursplan



Utbildningsmoment:

- Teoretiska och praktiska övningar ✓
- Webbkommunikation ✓
- Webservices ✓
- HTTP/HTTPS ✓
- JSON ✓
- XML
- Java ✓
- Databaser ✓
- Java applikationer ✓



01

ÖVERSIKT

Endpoints

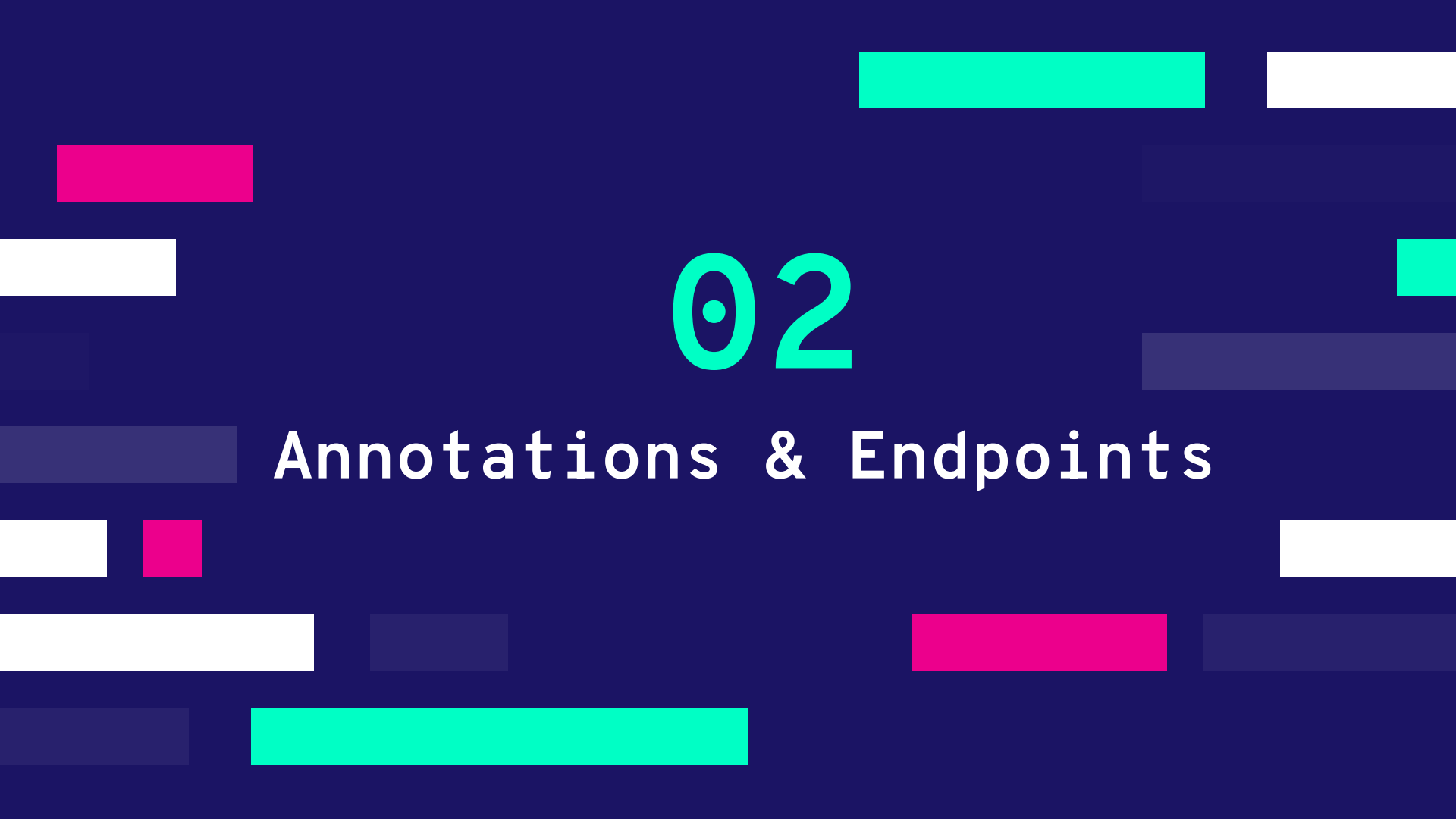


Endpoints?

Vad mer kan vi göra med 'endpoints'?

- Vad är en 'endpoint'?
- Vad klassas `{ }` inom 'URI'?
- 'Request Param' VS 'Query Param'
- 'GetMapping' VS 'RequestMapping'





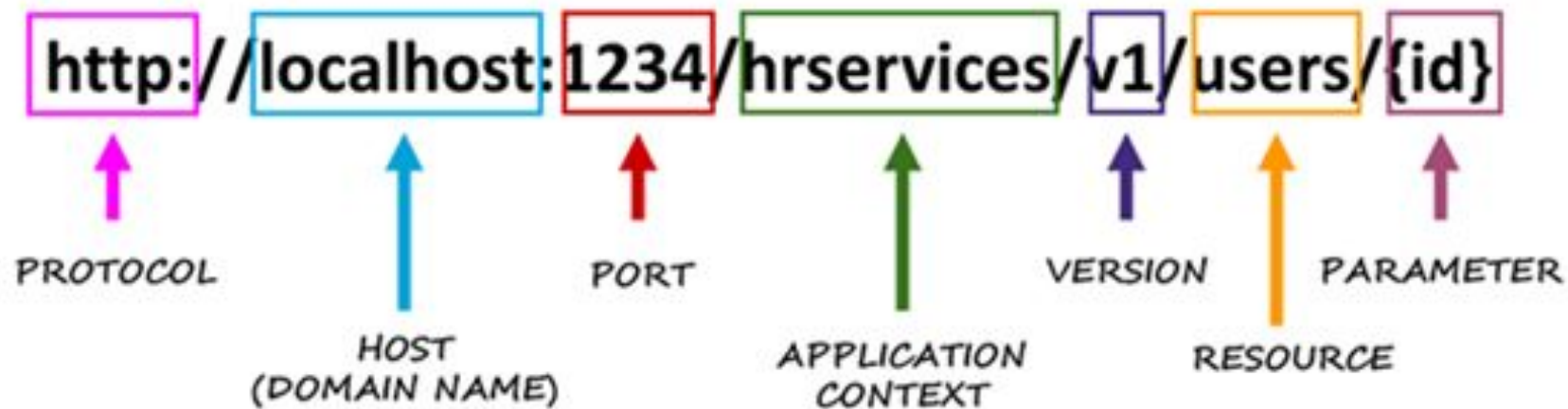
02

Annotations & Endpoints

URL Structure



RESTful ENDPOINT



RequestMapping VS GetMapping?



@RequestMapping

@GetMapping

Varför Två...?

När ni söker efter dokumentation
så kan ni ibland se att det finns
valmöjligheter mellan:

Get & RequestMapping

<https://www.logicbig.com/tutorials/spring-framework/spring-web-mvc/request-mapping-variants.html>



LATEST:

@GetMapping

Varför Två...?

Så simpelt är det...!

Ytterligare information:

<https://stackoverflow.com/questions/39077787/difference-between-the-annotations-getmapping-and-requestmappingmethod-requestmapping>



Mappings...?



Definitioner



Mapping()

En annotation

Följs av
GET, PUT, DELETE, POST



GET

GetMapping

Hämta värde

```
@GetMapping("/getTest")  
public void getTest() {  
  
}
```





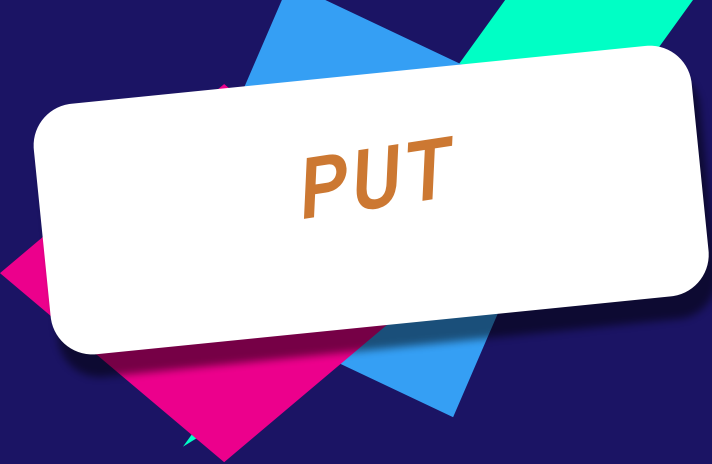
POST

PostMapping

Lägger upp ett nytt värde

```
@PostMapping("/postTest")  
public void postTest() {  
  
}
```





PUT

PutMapping

Ändrar ett befintligt värde

```
@PutMapping("/putTest")  
public void putTest() {  
  
}
```



A white rounded rectangle with the word "DELETE" in orange, tilted slightly to the right. It is positioned over a background of overlapping blue, cyan, and magenta triangles.

DELETE

DeleteMapping

Tar bort ett befintligt värde

```
@DeleteMapping("/deleteTest")  
public void deleteTest() {  
  
}
```

Be mindful about PATCH



Idempotency

Idempotency

- **PUT:** The **PUT** method is idempotent, meaning that applying the same **PUT** request multiple times will have the same effect as applying it once. This is important for reliability and consistency, especially in distributed systems where retries might occur.
- **PATCH:** **PATCH** is not guaranteed to be idempotent. Multiple identical **PATCH** requests might result in different outcomes depending on how the server processes them. This can lead to inconsistent states if the patch operation is not designed to be idempotent.

Idempotency

Source:

<https://stackoverflow.com/questions/28459418/use-of-put-vs-patch-methods-in-rest-api-real-life-scenarios/39338329#39338329>

To illustrate a non-idempotent PATCH, suppose there is a `/users` resource, and suppose that calling `GET /users` returns a list of users, currently:

```
[{ "id": 1, "username": "firstuser", "email": "firstuser@example.org" }]
```

Rather than PATCHing `/users/{id}`, as in the OP's example, suppose the server allows PATCHing `/users`. Let's issue this PATCH request:

```
PATCH /users
[{ "op": "add", "username": "newuser", "email": "newuser@example.org" }]
```

Our patch document instructs the server to add a new user called `newuser` to the list of users. After calling this the first time, `GET /users` would return:

```
[{ "id": 1, "username": "firstuser", "email": "firstuser@example.org" },
{ "id": 2, "username": "newuser", "email": "newuser@example.org" }]
```

Now, if we issue the *exact same* PATCH request as above, what happens? (For the sake of this example, let's assume that the `/users` resource allows duplicate usernames.) The "op" is "add", so a new user is added to the list, and a subsequent `GET /users` returns:

```
[{ "id": 1, "username": "firstuser", "email": "firstuser@example.org" },
{ "id": 2, "username": "newuser", "email": "newuser@example.org" },
{ "id": 3, "username": "newuser", "email": "newuser@example.org" }]
```

The `/users` resource has changed *again*, even though we issued the *exact same* PATCH against the *exact same* endpoint. If our PATCH is $f(x)$, $f(f(x))$ is not the same as $f(x)$, and therefore, **this particular PATCH is not idempotent**.

PATCH

Patch Mapping

Ändrar ett befintligt värde...
Och gör lite till?

```
@PatchMapping("/patchTest")  
public void patchTest() {  
  
}
```

Skillnader mellan PUT och
PATCH:
<https://stackoverflow.com/questions/28459418/use-of-put-vs-patch-methods-in-rest-api-real-life-scenarios>

PATCH - Less Syntax

```
PUT /users/1
{
  "username": "skwee357",
  "email": "skwee357@gmail.com"    // new email address
}
```

You can accomplish the same using PATCH. That might look like this:

```
PATCH /users/1
{
  "email": "skwee357@gmail.com"    // new email address
}
```


Tips



TIP

```
@DeleteMapping("/test")  
public void deleteTest() {}
```

```
@PostMapping("/test")  
public void postTest() {}
```

```
@GetMapping("/test")  
public void getTest() {}
```

Endpoints...?

Alla endpoints KAN ha
samma utgångspunkt.

*(Det går även att sätta en
endpoint på klass nivå)*

ERRORS



Webbplatser

Fråga: Vad är det för typer av 'requests' som görs när vi går in på en webbplats?

POST GET PUT
DELETE



ERROR?

Hantering

Delete, Post, Put kommer ej fungera via webbläsaren...

Det är här POSTMAN kommer in!

DOWNLOAD:
<https://www.postman.com/>

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Thu Nov 17 08:08:07 CET 2022

There was an unexpected error (type=Method Not Allowed, status=405).

RESTful + Spring



MODEL

```
public record User(long id, String name){  
  
}
```

Setup User

Vi gör samma setup som förra lektion - fast den här gången med en 'record'

Här ser vi att vi har ett 'ID' och ett 'name'

CONTROLLER

Setup Controller

Ta detta exempel med en
ArrayList av 'nya användare'

(Detta är bara för
DEBUGGING)

```
@RestController
public class UserController {

    List<User> userList = new ArrayList<> (Arrays.asList(
        new User(0, "Benny"),
        new User(1, "Benny"),
        new User(2, "Leif"),
        new User(3, "Anton"),
        new User(4, "Frida")
    ));
```


GET

Get all users

Hämta användare!

```
@GetMapping(value = "/users")  
public void getUsers() {  
  
    for (User user : userList) {  
        System.out.println(user);  
    }  
  
}
```

Enhanced for loop

(Fungerar med Stream eller alternativt for loop)

POST

Post 'user'

Lägg till användare!

```
@PostMapping(value = "/users")
public void createUser() {

    userList.add(
        new User(userList.size(), "Debugging")
    );
}
```

PathVariable

Pathvariable

```
@GetMapping(value = "/users/{id}")
public void getUser(@PathVariable("id") String id) {

    System.out.println(userList.get(Integer.parseInt(id)));

    for (User user : userList) {

        if (user.id() == Integer.parseInt(id)) {
            System.out.println(user);
        }
    }
}
```

Hämta användare baserat på ett ID

(Detta är endast till för debugging och kommer bli allt mer logiskt senare med Queries)

<https://www.baeldung.com/spring-controller-vs-restcontroller> Part 2

PathVariable DELETE

Delete User

```
@DeleteMapping("/{id}")
public User deleteUser(@PathVariable("id") int id) {

    for (User user : userList) {
        if (user.id() == id) {
            return userList.remove(id);
        }
    }

    return null;
}
```

Vi kan tekniskt sätt returnera 'void' här med, men det är ALLTID rekommenderat att försöka returnera 'något' i form av SVAR.

*RequestParam är ett
utmärkt val för när värden
inte är ett 'måste'*

RequestParam?

```
@GetMapping("/users")
public ResponseEntity<Page<User>> getUsersWithPagination(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "name") String sortBy) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));
    Page<User> users = userRepository.findAll(pageable);
    return ResponseEntity.ok(users);
}
```

Postman Install





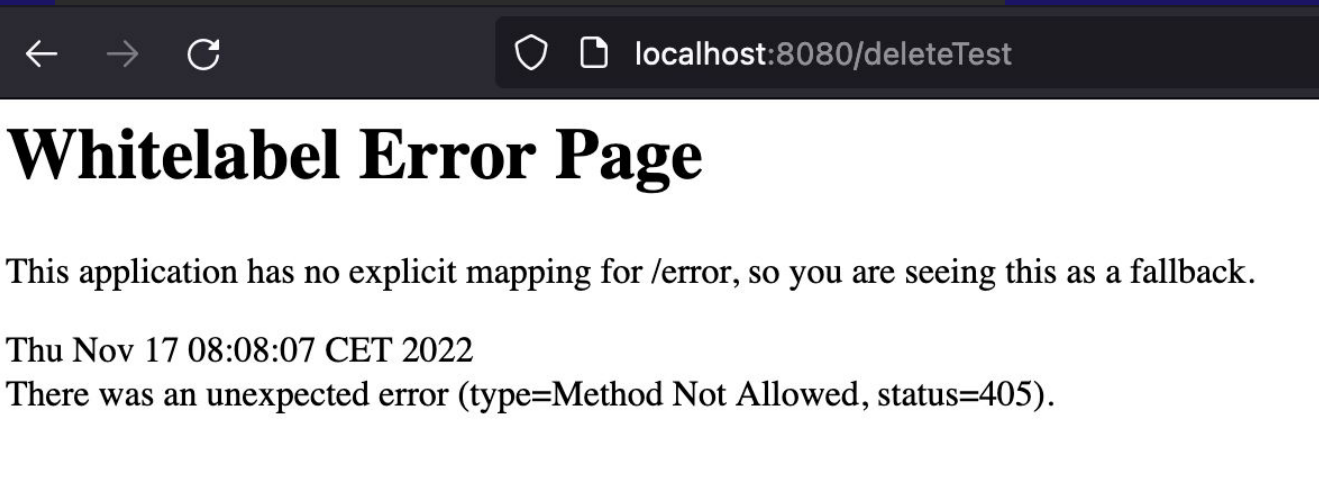
Recap

Hantering

Delete, Post, Put kommer ej fungera via webbläsaren...

Det är här POSTMAN kommer in!

DOWNLOAD:
<https://www.postman.com/>



Om ni har mac och undrar om ni ska välja 'Intel' eller 'Apple' så gå in här:



Om ni har intel kommer det stå 'intel' på 'CHIP'

userInterface



My Workspace

New

Import

+ ...

No Environment

⌵

Collections

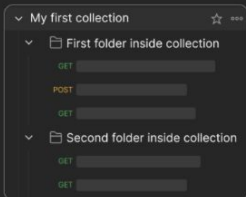
APIs

Environments

Mock Servers

Monitors

...



Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

Start working with APIs

⌵ ✕



67% complete - Nice work!

✓ Create account

✓ Send your first API request

○ Save requests

So you can avoid losing your progress and can share your work with people

[Show me](#)



Open Workspace Overview

Create a new request:

GET

POST

PUT

My Workspace

New

Import

+ ...

No Environment

...

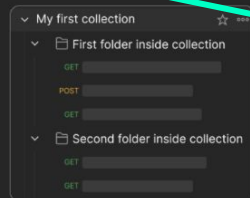
Collections

APIs

Environments

Mock Servers

Monitors



Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

Start working with APIs



67% complete - Nice work!

- ✓ Create account
- ✓ Send your first API request

- ☐ Save requests
So you can avoid losing your progress and can share your work with people
[Show me](#)



Open Workspace Overview

Create a new request:



My Workspace

New

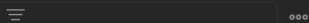
Import



No Environment



Collections



APIs



Environments



Mock Servers



Monitors



My first collection ☆

First folder inside collection

GET

POST

GET

Second folder inside collection

GET

GET

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

Start working with APIs



67% complete - Nice work!

- ✓ Create account
- ✓ Send your first API request

- ☐ Save requests
So you can avoid losing your progress and can share your work with people
[Show me](#)

Tryck på +



Open Workspace Overview

Create a new request:



Setting Up Request

GET Untitled Request + ... No Environment

Untitled Request Save ...

GET Enter request URL Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
Key	Value	Description		

Setting Up Request

GET Untitled Request

+

...

Untitled Request

GET 

Enter request URI

Params Authorization Headers (6) Body Pre-request Script

Query Params

	KEY	VALUE
	Key	Value

Vi kan ändra på 'GET' till

- Post
- Put
- Patch
- Delete

Setting Up Request

DELETE	▼	http://localhost:8080/deleteFirstUser	Send	▼
--------	---	---------------------------------------	------	---

Skriv in er URL + Endpoint och testa den från Postman!





04

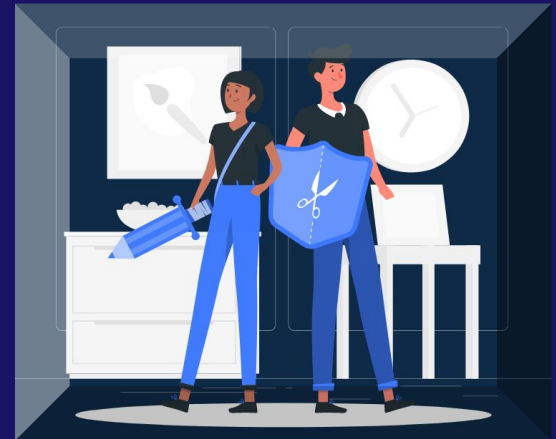
Uppgifter
&
Eget Arbete

Välkommen till Uppgifterna!

Uppgifterna är till för att testa dina färdigheter och kunskaper för att både öva och repetera på det vi har arbetat med under föreläsningarna.

Dessa är **INTE** obligatoriska.
Men är **starkt rekommenderat** att arbeta med!

Uppgifter



```
1          // -Uppgift #1- //
2
3      /* INSTRUCTIONS
4
5         Skapa ett nytt projekt med Spring Init
6         WebService_Uppgifter_Lektion_4
7
8         Dependency: Spring Web
9
10     */
11
12     // HINT & Examples
13     hint(" https://start.spring.io/ ")
14
15
16
17
18
19
20
21
22
23
```

Uppgift #1

*Getting started is all about
doing the 'basics' first...*

```

1          // -Uppgift #2- //
2
3      /* INSTRUCTIONS
4
5          Skapa en ny 'Record' klass, döp den till
6          'user'
7
8          Definiera användarens attribut:
9              • Id
10             • Username
11             • Password
12             • IsEnabled
13
14      */
15
16      // HINT & Examples
17      hint(" Package för denna klass kan ligga som
18          'model' ")
19
20
21
22
23

```

Uppgift #2

*Tired of creating Getters &
Setters and overrides?
Try Records!*

Records shorthand syntax are
interesting and a warm welcome
to the land of java!

```

1      // -Uppgift #3- //
2
3      /* INSTRUCTIONS
4
5         Åkalla nu din 'User' inom en
6         'restController getMapping'
7
8         Du kan skicka vidare din nya användare
9         direkt inom ett 'return statement'
10
11        Return new User(0, "Benny", "123", true)
12
13        */
14
15        // HINT & Examples
16        hint(" Glöm inte att använda rätt
17        annotationer och returtyper")
18
19        hint(" Du vet att du är klar när du kan se
20        din nya användare på:
21        localhost:8080/newUser ")
22
23

```

Uppgift #3

On the next task we'll be working with pathvariables for dynamic values!

```

1          // -Uppgift #4- //
2
3      /* INSTRUCTIONS
4
5          Inom din @GetMapping metod, använd nu
6          av @PathVariable för att definiera ett
7          Användarnamn.
8
9          Returtypen hade kunnat se ut så här:
10
11          Return new User(0, username, "123", true)
12
13      */
14
15      // HINT & Examples
16      hint(" PathVariables definieras inom metodens
17      parametrar:
18
19          public void test(@PathVariable String username) {}
20
21      ")
22
23

```

Uppgift #4

PathVariables are great for inserting dynamic data, especially during our posts!

```
1          // -Uppgift #5- //
2
3      /* INSTRUCTIONS
4
5          Definiera en ArrayList med CustomUsers
6
7          Definiera en @DeleteMapping
8
9          Använd dig av Postman för att debugga
10
11      */
12
13
14
15
16
17
18
19
20
21
22
23
```

Uppgift #5

RÄTT ELLER FEL

1. **HTTP**, är ett protokoll
2. **URL**, inkluderar ett domännamn
3. **API & Web Service**, kan inkludera versionsnummer inom sin **URI**
4. **Parametrar** hjälper användare att navigera till statiska resurser

Markera texten
med muspekaren
för att se svar!

RÄTT ELLER FEL

1. **URI**, är ett sätt för oss att definiera 'endpoints'
2. **Navigering** in mot en webbplats kör alltid igång en PUT request
3. **@GetMapping**, går att definiera på klassnivå
4. **PathVariable**, indikerar att parametern är kritisk och måste användas för att identifiera resurser

Markera texten
med muspekaren
för att se svar!

Fördjupningsfråga

Varför är det alltid rekommenderat att returnera ett svar inom vår mapping?

Tänk om vi har kod där vi tekniskt sett inte behöver returnera något..

Vi sparar även på kod & tydlighet om vi tar bort detta.. eller?

```
@DeleteMapping("/{id}")
public User deleteUser(@PathVariable("id") int id) {

    for (User user : userList) {
        if (user.id() == id) {
            return userList.remove(id);
        }
    }

    return null;
}
```

Svar på nästa
sida

FACIT

`null` är inte informativt för klienten.

Använd hellre 404 när något inte hittas, och välj om du vill returnera den borttagna resursen (200 OK) eller bara status utan body (204 No Content).

Välkommen till KOD UTMANINGAR!

Här kommer du få olika kodstycken som du skall kopiera och klistra in till din app, för att sedan försöka felsöka och lösa.

Här får du övat på:

- * Felsökning & Felhantering
- * Kod Förståelse

Försök undvika CHATgpt då detta förstör för själva lärandet!

CODE CHALLENGE



```
1  @RestController
2  @RequestMapping("/users")
3  public class UserController {
4
5      private List<CustomUser> userList;
6
7      @GetMapping()
8      public List<CustomUser> getAllUsers() {
9
10         return userList;
11     }
12
13 }
```

Uppgift #1

Copy & Paste the code

*Will this run: **why / why not?***

Then try to run the app and see!

```
1 MARKERA FÖR SVAR:
2
3 #1 - Returtyp
4
5 #2 - Null
6
7 #3 - Sen instansiering
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
```

HINTS #1

Facit existerar på nästkommande sida

FACIT:

Det går utmärkt! :)

FACIT #1

MARKERA TEXTEN FÖR SVAR

```
1  @RestController
2  @RequestMapping("/users")
3  public class UserController {
4
5      private final List<User> userList;
6
7      @GetMapping()
8      public List<User> getAllUsers() {
9
10         return userList;
11     }
12
13     @PostMapping("/{username}")
14     public boolean postUser(@PathVariable
15 String username) {
16
17         return userList.add(
18             new User(userList.size(),
19 username)
20         );
21     }
22
23 }
```

Uppgift #2

Copy & Paste the code

Try to run the app and see if it works.

Why / Why not?

```
1 MARKERA FÖR SVAR:
2
3 #1 - Initialisering
4
5 #2 - Tom Lista
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
```

HINTS
Upg. #2

Facit existerar på nästkommande sida

FACIT:

OM en inte har en konstruktor, så kan en
fylla på med en tom lista

Som så:

```
private final List<User> userList = new  
ArrayList<>(100);
```

FACIT
Upg. #2

MARKERA TEXTEN FÖR SVAR

```
1  @PostMapping("/add")
2  public void addUser(@RequestParam String
3  username) {
4
5      User user = new User(userList.size(),
6  username);
7      userList.add(user);
8      return user;
9  }
```

Uppgift #3

Copy & Paste the code

Try to run the app and see if it works.

Why / Why not?

1 MARKERA FÖR SVAR:

2

3 #1 - Datatyp

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

HINTS
Upg. #3

*Facit existerar på nästkommande
sida*

1 **FACIT:**
2
3 Returnerar vi en Användare, måste det även
4 specificeras inom metoden.
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23

FACIT
Upg. #3

MARKERA TEXTEN FÖR SVAR

```
1  @RestController
2  @RequestMapping("/users")
3  public class UserController {
4
5      private final List<User> userList;
6
7      // Constructor to initialize the
8      userList with users 1 to 10
9      public UserController() {
10         this.userList = new ArrayList<>();
11         for (int i = 1; i ≤ 10; i++) {
12             userList.add(new User(i, "User" +
13 i));
14         }
15     }
16
17     @GetMapping()
18     public List<User> getAllUsers() {
19         return userList;
20     }
21 }
22
23
```

Uppgift #4

PART #1

```
1  @PostMapping("/add")
2  public boolean addUser(@RequestParam int
3  userId, @RequestParam String username) {
4
5      return userList.add(
6      new User(userId, username)
7      );
8  }
9
10
11  /* Todo,
12
13      #1 - Users with same id's shouldn't
14          be created
15
16      #2 - Throw an error if the user isn't
17          Successfully created.
18
19      #3 - Create an easy auto generated unique
20          ID.
21
22
23
```

Uppgift #4

PART #2

MARKERA FÖR SVAR:

#1 - Users with same id's shouldn't be created

HINT: For loop if-check

#2 - Throw an error if the user isn't Successfully created.

HINT: If-else with throw

#3 - Create an easy auto generated unique ID.

HINT: id = arraylist.size

HINTS
Upg. #4

Facit existerar på nästkommande sida

FACIT UTAN förenklad kod:

```
1  @PostMapping("/add")
2
3
4  public boolean addUser(@RequestParam int userId, @RequestParam String
5  username) {
6
7      try {
8          // Check if the userId already exists in the userList
9          for (User user : userList) {
10
11              if (user.getId() == userId) {
12                  throw new IllegalArgumentException("User ID " + userId +
13                  " already exists.");
14              }
15          }
16
17          // If no exception is thrown, add the new user
18          return userList.add(new User(userId, username));
19      } catch (IllegalArgumentException e) {
20          // Handle the exception (e.g., log it, rethrow it, etc.)
21          System.err.println(e.getMessage());
22
23          return false;
24      }
25  }
```

FACIT Upg. #4

MARKERA TEXTEN FÖR SVAR

1 FACIT med förenklad kod:

2 @PostMapping("/add")

3
4 public boolean addUser(@RequestParam String
5 username) {

6 // Generate a new unique ID for the user

7
8 int newUserId = userList.size() + 1;

9
10
11
12
13 // Add the new user with the generated
14 unique ID

15
16 return userList.add(new User(newUserId,
17 username));

18
19 }
20
21
22
23

0 1 0 1 0 0
0 1 0 1 0
0 1
0

FACIT
Upg. #4

0 1 0 1
1 0 0 0 0 0 0 1 0 1
0 1 0 0
0 0
0

MARKERA TEXTEN FÖR SVAR

THANKS!

Do you have any questions?
kristoffer.johansson@sti.se

sti.learning.nu/

CREDITS: This presentation template was created by
Slidesgo, including icons by Flaticon, and
infographics & images by Freepik.

You can also contact me VIA Teams (quicker response)
Du kan också kontakta mig VIA Teams (Snabbare svar)